DeepSeek

Scaling Law

深入研究背景：为什么需要重新审视Scaling Law?

尽管Scaling Law被奉为LLM领域的“第一性原理”，但DeepSeek团队在开篇就指出了一个关键矛盾：**不同顶尖研究团队得出的最优缩放策略存在显著差异**。

- **经典研究的矛盾**：
  - **OpenAI** 在2020年的研究中指出，在计算预算增加时，模型参数的增长应优先于数据量的增长。
  - **DeepMind (Chinchilla)** 在2022年的研究却得出了相反的结论：模型普遍“训练不足”，在相同计算预算下，应该训练一个参数更少但数据更多的模型。
- **问题的根源**：DeepSeek认为，这些差异可能源于：
  1. **超参数设置**：早期的研究可能没有在每种计算规模下都找到真正最优的超参数（如Batch Size、学习率），导致模型未达到最佳性能，从而影响了缩放策略的结论。
  2. **模型规模度量不精确**：传统使用模型参数量 `N` 并套用 `C ≈ 6ND` 的公式来估算计算量，这是一个**严重有偏的近似**，因为它忽略了Transformer中注意力机制的计算开销。
  3. **数据质量的影响**：不同研究使用的数据质量不同，而数据质量被普遍认为是关键因素，但其对缩放策略的定量影响尚未被系统研究。

DeepSeek的**长期主义** 就体现在这里：他们不急于堆砌参数刷榜，而是选择先**“拨开乌云”**，从根本上理解和修正Scaling Law，确保后续的每一步扩展都在可预测、最优的路径上。

模型架构：不止于LLaMA的复现

DeepSeek LLM的架构在LLaMA的基础上，做出了一个**战略性的结构选择**。

- **基础设定**：
  - **Decoder-Only**：标准的自回归Transformer解码器架构。
  - **Pre-LayerNorm**：使用RMSNorm进行层归一化，提升训练稳定性。
  - **激活函数**：使用SwiGLU激活函数，替代传统的ReLU或GELU，以提升性能。
  - **位置编码**：使用Rotary Positional Embedding (RoPE)，更好地处理长序列。
- **关键创新：深度优先的缩放策略**
  - **常规做法**：扩大模型规模时，通常会同比例增加层数（深度）和隐藏层维度（宽度）。
  - **DeepSeek的选择**：他们**显著地增加了网络的深度**。
    - **DeepSeek-LLM 7B**：30层。
    - **DeepSeek-LLM 67B**：95层。
  - **动机**：他们假设通过增加深度可以更有效地提升模型的表示能力和层次化特征提取能力。这与通常增加FFN中间层宽度的方式形成对比，体现了他们对模型结构空间的深入探索。
- **推理优化**：
  - 67B模型采用了**分组查询注意力（GQA）**。这是一种在多头注意力（MHA）和多查询注意力（MQA）之间的折中方案。它将查询头（Query Heads）分组，每组共享同一套键（Key）和值（Value）头。这能在几乎不损失模型性能的前提下，大幅减少推理时KV缓存的内存占用，从而提升推理速度。

Scaling Law的核心研究：方法论与发现

这是论文最硬核的部分，分为三个环环相扣的章节。

研究一：超参数的Scaling Law——为模型找到“最佳工作点”

目标：确定在不同计算预算 `C` 下，最优的批处理大小（Batch Size, `B`）和学习率（Learning Rate, `η`）。

方法：

1. **初步探索**：在较小的计算规模（177M FLOPs/token）上进行网格搜索，发现**泛化误差在一个很宽的超参数范围内保持稳定**（变化不超过0.25%）。这意味着找到一个“足够好”的超参数组合相对容易，但找到一个“精确最优”的点则非常困难且不必要。
2. **系统实验**：在从 `1e17` 到 `2e19` FLOPs的多个计算规模上，训练了大量模型，每个规模下尝试不同的 `(B, η)` 组合。
3. **定义“近似最优”**：他们将所有**泛化误差不超过观测到的最小误差0.25%** 的超参数点，定义为“近似最优”超参数。
4. **曲线拟合**：对这些“近似最优”区域中的典型值进行幂律关系拟合。

核心发现与公式：

- **最佳批处理大小** `B_opt` 随计算预算 `C` 的增加而**增长**。
- **最佳学习率** `η_opt` 随计算预算 `C` 的增加而**下降**。
- 他们得出了**量化的预测公式**：

$$\eta_{opt} = 0.3118 \cdot C^{-0.1250}$$
$$B_{opt} = 0.2920 \cdot C^{0.3271}$$

  意义：这套公式使得DeepSeek在面对任何新的计算预算时，都能快速锁定一个高性能的超参数起点，极大减少了调参成本，并确保了后续Scaling Law研究的基线性能是最优的。

研究二：模型规模的精确度量与最优分配——纠正历史偏差

目标：找到在给定计算预算 `C` 下，模型规模 `M` 和数据规模 `D` 的最优分配比例。

问题：传统度量 `N` （参数量）和公式 `C ≈ 6ND` 不准确。

DeepSeek的革新：

- **提出新度量** `M`：**非嵌入层 FLOPs/token**。这是一个更精确的、前向传递一次所消耗的计算量。
  - **计算公式**：

$$M = \underbrace{72\, n_{layer}\, d_{model}^2}_{\text{FFN和注意力中的线性层}} + \underbrace{12\, n_{layer}\, d_{model}\, l_{seq}}_{\text{注意力计算（QK和PV）}}$$

    其中 `n_layer` 是层数，`d_model` 是隐藏层维度，`l_seq` 是序列长度。
- **更新计算预算公式**：`C = M * D`

为什么 `M` 更好?
论文通过一个详尽的表格对比了不同规模模型下 `6N1`，`6N2` 和 `M` 的比值：

模型配置	$\frac{6N_1}{M}$	$\frac{6N_2}{M}$
小模型 (e.g., 8层)	0.43 (低估57%)	1.32 (高估32%)
大模型 (e.g., 80层)	0.92 (低估8%)	0.94 (低估6%)

- 可见，对于**小模型**，传统度量 `6N1` **严重低估**了实际计算量，而 `6N2` **严重高估**了计算量。这会导致在拟合Scaling Law曲线时，在小规模阶段就引入巨大误差，进而影响对大模型性能的预测。

IsoFLOPs 实验与发现：

- **方法**：为了在可控成本下研究大计算预算的行为，他们采用了 **IsoFLOPs** 方法。即固定一个计算预算 `C`，然后训练一系列具有不同 `M/D` 比例（即不同模型架构但总计算量相同）的模型，绘制出损失曲线。
- **成果**：
  1. 通过多个不同 `C` 下的IsoFLOPs实验，他们拟合出了**最优模型规模** `M_opt` 和**最优数据规模** `D_opt` 与计算预算 `C` 的幂律关系：
     $M_{opt} = 0.1715 \cdot C^{0.5243}$
     $D_{opt} = 5.8316 \cdot C^{0.4757}$
  2. **核心结论**：在给定计算预算 `C` 下，最优的策略是**将约52.4%的预算分配给模型规模，47.6%的预算分配给数据规模**。这一定量结论介于OpenAI和Chinchilla之间，但因其更精确的度量而更具普适性。
  3. **预测验证**：使用此公式预测的DeepSeek LLM 7B和67B的最终性能（损失）与真实训练结果高度吻合，证明了其预测能力。

研究三：数据质量的量化影响——一个关键调节变量

洞察：最优的 `M/D` 分配比例并非一成不变，它受到**数据质量**的显著影响。

方法：在三个不同质量的数据集上重复了上述Scaling Law分析：

1. **早期内部数据**：质量较低。
2. **当前内部数据**：质量较高。
3. **OpenWebText2**：一个公认的、经过严格清洗的高质量开源数据集。

惊人发现：

- 随着数据质量的提升，最优的**模型缩放指数** `a` 逐渐**增大**，而**数据缩放指数** `b` 逐渐**减小**。

数据集	模型缩放指数 a	数据缩放指数 b
OpenAI (OpenWebText2)	0.73	0.27
Chinchilla (MassiveText)	0.49	0.51
Ours (Early Data)	0.450	0.550
Ours (Current Data)	0.524	0.476
Ours (OpenWebText2)	**0.578**	**0.422**

- **解释**：高质量数据信息密度高、噪声少，模型可以从中学习到更丰富、更精确的模式。因此，**提供一个更强大的模型（更大的 `M`）来消化和利用这些高质量信息是更有效的**。相反，低质量数据包含大量冗余和噪声，单纯增加模型参数容易过拟合或学不到新东西，此时增加数据量（更大的 `D`）进行充分训练更为重要。

意义：这一发现**统一并解释了**OpenAI和Chinchilla结论不一致的原因——他们使用的数据质量不同。这也为整个行业提供了一个重要指南：**当你提升数据质量时，你的模型缩放策略也应该随之动态调整。**

从基座模型到对话模型：SFT与DPO实战

- **数据构建**：收集了约150万条中英文指令数据，分为：
  - **有用性数据 (1.2M)**：涵盖通用语言任务(31.2%)、数学问题(46.6%)和代码练习(22.2%)。
  - **安全性数据 (0.3M)**：涵盖各种敏感话题，用于训练模型拒绝有害请求。
- **两阶段对齐**：
  1. **监督微调（SFT）**：
     - 发现**大模型更容易过拟合**。因此7B模型训练了4轮，而67B模型只训练了2轮。
     - 发现了一个有趣的现象：在数学数据上，**能力较弱的模型（7B）更容易陷入“循环重复”的生成**。他们认为这是因为数学解题模式本身具有一定重复性，弱模型会过度模仿这种表面模式。
  2. **直接偏好优化（DPO）**：
     - 为了解决SFT后的重复等问题，并进一步对齐人类偏好，他们采用了DPO。
     - 他们构建了偏好数据集，使用DeepSeek Chat模型生成对同一提示词的不同响应，并由标注员选出更优的答案。
     - **效果**：DPO训练有效地减轻了重复响应问题，增强了回答的多样性和连贯性，同时没有损害模型在标准Benchmark上的能力。

MOE

背景：Transformer 中的混合专家（MoE）基础

标准 Transformer 语言模型由 L 层堆叠而成，每层包含自注意力模块和前馈网络（FFN）：

$$\begin{aligned}\mathbf{u}_{1:T}^{l} &= \text{Self-Att}\left(\mathbf{h}_{1:T}^{l-1}\right) + \mathbf{h}_{1:T}^{l-1}, \\ \mathbf{h}_{t}^{l} &= \text{FFN}\left(\mathbf{u}_{t}^{l}\right) + \mathbf{u}_{t}^{l},\end{aligned}$$

其中 $\mathbf{h}_t^l \in \mathbb{R}^d$ 为第 t 个 token 在第 l 层的输出（省略 LayerNorm）。

在 MoE 架构中，部分或全部 FFN 被替换为**混合专家层**。该层包含 N 个结构相同的专家（每个即一个 FFN），每个 token 通过门控机制被路由至 K 个专家（如 Top-1 或 Top-2）：

$$\begin{aligned}\mathbf{h}_t^l &= \sum_{i=1}^{N} g_{i,t}\,\text{FFN}_i(\mathbf{u}_t^l) + \mathbf{u}_t^l, \\ g_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} \in \text{TopK}(\{s_{j,t}\}_{j=1}^{N}, K), \\ 0, & \text{否则}, \end{cases} \\ s_{i,t} &= \text{Softmax}_i\left((\mathbf{u}_t^l)^\top \mathbf{e}_i^l\right),\end{aligned}$$

其中 $\mathbf{e}_i^l$ 为第 i 个专家的路由中心向量。由于 $g_{i,t}$ 稀疏（仅 K 个非零），MoE 在保持大模型容量的同时控制计算开销。

架构设计

在通用 MoE 基础上引入两项核心策略，旨在**最大化专家专业化程度**，同时保持总参数量与计算成本不变：

> **目标**：让每个专家专注于特定知识子集，避免冗余学习，提升模型表达效率。

细粒度专家分割（Fine-Grained Expert Segmentation）

动机

当专家数量有限时，单个专家需处理多样化的 token，导致其内部参数被迫编码异构知识，难以高效利用。

方法

- 将每个原始专家 FFN 沿中间维度**均匀切分为 m 个小专家**（中间维度缩小为原来的 $1/m$）；
- 同时将激活专家数从 K 提升至 mK，以维持相同计算量；
- 总专家数从 N 扩展至 mN。

此时 MoE 输出变为：

$$\mathbf{h}_t^l = \sum_{i=1}^{mN} g_{i,t}\,\mathrm{FFN}_i(\mathbf{u}_t^l) + \mathbf{u}_t^l, \quad g_{i,t} \neq 0 \text{ for } mK \text{ experts.}$$

优势：组合灵活性剧增

- 例：$N = 16, K = 2 \to$ 原始组合数：$\binom{16}{2} = 120$；
- 若 $m = 4$，则 $mN = 64, mK = 8 \to$ 组合数：$\binom{64}{8} \approx 4.4 \times 10^9$；
- 更丰富的专家组合使模型能更精准地匹配 token 的语义需求，实现**细粒度知识分配**。

共享专家隔离（Shared Expert Isolation）

动机

不同专家可能重复学习通用知识（如语法、常见事实），造成参数冗余。

方法

- 额外指定 K_s 个专家作为**共享专家**；
- **所有 token 强制激活这些共享专家**；
- 为保持计算总量不变，动态减少路由专家的激活数：从 mK 降至 $mK - K_s$。

完整输出公式为：

$$\mathbf{h}_t^l = \underbrace{\sum_{i=1}^{K_s} \mathrm{FFN}_i(\mathbf{u}_t^l)}_{\text{共享专家}} + \underbrace{\sum_{i=K_s+1}^{mN} g_{i,t}\,\mathrm{FFN}_i(\mathbf{u}_t^l)}_{\text{路由专家}} + \mathbf{u}_t^l,$$

其中路由仅在后 $mN - K_s$ 个专家中进行 Top-$(mK - K_s)$ 选择。

效果

- 共享专家集中建模通用模式；
- 路由专家可更专注地学习**差异化、专业化知识**；
- 整体参数效率提升。

> 注：该思想受 DeepSpeed-MoE 启发，但本文从**算法设计角度**出发，强调专业化而非工程优化。

负载均衡机制

自动路由易导致**负载不均衡**，引发两大问题：

1. **路由坍塌（Routing Collapse）**：少数专家被频繁使用，其余未充分训练；
2. **设备计算瓶颈**：专家分布不均导致某些设备过载。

为此，引入双重平衡损失：

专家级平衡损失（Expert-Level Balance Loss）

防止路由坍塌：

$$\begin{aligned}\mathcal{L}_{\text{ExpBal}} &= \alpha_1 \sum_{i=1}^{N'} f_i P_i, \\ f_i &= \frac{N'}{K'T} \sum_{t=1}^{T} (\text{Token } t \text{ selects Expert } i), \\ P_i &= \frac{1}{T} \sum_{t=1}^{T} s_{i,t},\end{aligned}$$

其中 $N' = mN - K_s, K' = mK - K_s$，$\alpha_1$ 为平衡因子（设为较小值）。

设备级平衡损失（Device-Level Balance Loss）

优化分布式训练效率：

- 将路由专家划分为 D 组，每组部署于一个设备；
- 对每组计算平均负载：

$$\begin{aligned}\mathcal{L}_{\text{DevBal}} &= \alpha_2 \sum_{i=1}^{D} f_i' P_i', \\ f_i' &= \frac{1}{|\mathcal{E}_i|} \sum_{j \in \mathcal{E}_i} f_j, \quad P_i' = \sum_{j \in \mathcal{E}_i} P_j.\end{aligned}$$

- α_2 设为较大值，优先保障设备间计算均衡，避免性能瓶颈。

Group Relative Policy Optimization (GRPO)

背景：从 PPO 到 GRPO

Proximal Policy Optimization (PPO) 是 LLM 强化学习（RL）微调中广泛使用的算法。其目标函数为：

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}_{q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)} \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_\theta(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t,\ \text{clip}\left(\cdot, 1-\epsilon, 1+\epsilon\right) A_t \right]$$

其中：

- A_t 为优势函数，通过 GAE 基于奖励序列和值函数 V_ψ 计算；
- 奖励通常包含 KL 正则项：

$$r_t = r_\phi(q, o_{\leq t}) - \beta \log \frac{\pi_\theta(o_t|q, o_{<t})}{\pi_{ref}(o_t|q, o_{<t})}$$

问题：PPO 需训练一个与策略模型规模相当的值函数模型，带来显著内存和计算开销；且在仅对最终 token 打分的场景下，逐 token 的值函数难以准确训练。

GRPO 核心思想

Group Relative Policy Optimization (GRPO) 提出一种无需值函数的替代方案：

- 对每个问题 q，从旧策略 $\pi_{\theta_{old}}$ 中采样一组输出 $\{o_1, \ldots, o_G\}$；
- 使用该组输出的**平均奖励作为基线**，计算相对优势 $\hat{A}_{i,t}$；
- 目标函数为：

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{q\sim P(Q),\{o_i\}\sim\pi_{\theta_{old}}(O|q)} \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|} \left\{\min\left[\frac{\pi_\theta}{\pi_{\theta_{old}}}\hat{A}_{i,t},\ \text{clip}(\cdot)\hat{A}_{i,t}\right] - \beta\mathbb{D}_{KL}[\pi_\theta\|\pi_{ref}]\right\}$$

- **KL 散度正则项**直接加入损失函数（而非奖励），使用无偏估计器：

$$\mathbb{D}_{KL}[\pi_\theta\|\pi_{ref}] = \frac{\pi_{ref}}{\pi_\theta} - \log\frac{\pi_{ref}}{\pi_\theta} - 1 \geq 0$$

优势：省去值函数训练，降低资源消耗；更契合奖励模型基于“比较”的本质。

GRPO 的两种监督模式

结果监督（Outcome Supervision）

- 每个输出 o_i 得到一个最终奖励 r_i；
- 组内标准化：$\tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$；
- 所有 token 的优势设为相同：$\hat{A}_{i,t} = \tilde{r}_i$。

过程监督（Process Supervision）

- 奖励模型对每个推理步骤打分，得到序列 $\{r_i^{index(j)}\}$；
- 全局标准化后，token 级优势为后续所有步骤奖励之和：

$$\hat{A}_{i,t} = \sum_{index(j)\geq t} \tilde{r}_i^{index(j)}$$

实验表明：**过程监督（GRPO+PS）优于结果监督（GRPO+OS）**。

迭代式 RL 训练（Iterative GRPO）

为应对奖励模型随策略进化而过时的问题，采用**迭代训练机制**：

1. 用当前策略采样新数据；
2. 通过**回放机制**（含 10% 历史数据）持续更新奖励模型；
3. 将当前策略设为参考模型，继续训练。

实验显示：**第一轮迭代即带来显著性能提升**。

关键发现与讨论

为何 RL 有效?

- **Pass@K 未提升，但 Maj@K 显著提升** → RL 并未增强基础能力，而是**优化输出分布**，使正确答案更可能出现在 Top-K 中；
- 与 [Wang et al., 2023] 观点一致：SFT 模型存在**推理任务中的偏好错位（misalignment）**，RL 可通过偏好对齐纠正。

如何实现更有效的 RL？（三大方向）

数据源（Data Source）

- 当前仅用 SFT 问题 + 简单采样 → 限制泛化；
- **未来方向**：
  - 引入**分布外问题**；
  - 采用**高级采样策略**（如树搜索）；
  - 结合**高效推理技术**（如推测解码）提升探索效率。

算法（Algorithm）

- 当前方法完全信任奖励信号，但奖励可能含噪（如 PRM800K 约 20% 标注错误）；
- **未来方向**：开发**对噪声鲁棒的 RL 算法**，探索 **Weak-to-Strong 对齐**范式。

奖励函数（Reward Function）

- 三个关键挑战：
  1. **提升泛化能力**：处理 OOD 问题与复杂生成；
  2. **建模不确定性**：作为弱奖励与强学习间的桥梁；
  3. **构建高质量过程奖励模型**：提供细粒度监督信号。

附录：各类方法统一视角（梯度系数分析）

方法	数据来源	奖励函数	梯度系数 GC
SFT	标注数据	人工选择	1
RFT	SFT 采样输出	规则（答案正确性）	$\mathbb{I}(o)$
Online RFT	实时策略采样	规则	$\mathbb{I}(o)$
DPO	SFT 采样对比对	人类偏好 / 规则	$\sigma(\Delta \log \pi/\pi_{ref})$
PPO	策略采样	奖励模型	A_t（依赖值函数）
GRPO	策略采样（组）	奖励模型	$\hat{A}_{i,t} + \beta(\pi_{ref}/\pi_\theta - 1)$

GRPO 的核心创新：**用组内相对奖励替代值函数，实现高效、稳定的优势估计**。

Multi-head Latent Attention（MLA）

目标：在保持甚至超越 MHA 性能的同时，大幅降低推理时的 KV 缓存开销。

背景与动机

- **标准 MHA**：每个 token 需缓存 $2n_h d_h l$ 个元素（l 为层数），成为推理瓶颈。
- **MQA / GQA**：通过共享 Key/Value 减少缓存，但性能下降。
- **MLA 方案**：引入**低秩联合压缩**，实现 **"更小缓存 + 更强性能"**。

核心机制

低秩 KV 联合压缩

- 将原始隐藏状态 $\mathbf{h}_t$ 投影为低维潜在向量：

$$\mathbf{c}_t^{KV} = W^{DKV}\mathbf{h}_t \in \mathbb{R}^{d_c}, \quad (d_c \ll d_h n_h)$$

- 从中恢复压缩后的 Key/Value：

$$\mathbf{k}_t^C = W^{UK}\mathbf{c}_t^{KV}, \quad \mathbf{v}_t^C = W^{UV}\mathbf{c}_t^{KV}$$

- **推理优势**：
  - 仅需缓存 $\mathbf{c}_t^{KV}$，KV 缓存降至 $d_c l$
  - 矩阵 W^{UK}、W^{UV} 可分别吸收到 W^Q、W^O 中，**无需显式计算 K/V**

Query 压缩（训练优化）

- 为减少激活内存，对 Query 同样进行低秩压缩：

$$\mathbf{c}_t^Q = W^{DQ}\mathbf{h}_t, \quad \mathbf{q}_t^C = W^{UQ}\mathbf{c}_t^Q$$

- 此操作不影响 KV 缓存，仅优化训练显存。

解耦 RoPE（Decoupled Rotary Position Embedding）

- **问题**：RoPE 与 KV 压缩不兼容——若对压缩后的 Key 应用 RoPE，会破坏矩阵吸收，导致必须重算历史 Key。
- **解决方案**：引入独立的 RoPE 分支：
  - 额外计算 RoPE-aware 查询 $\mathbf{q}_t^R = \mathrm{RoPE}(W^{QR}\mathbf{c}_t^Q)$
  - 共享 RoPE-aware 键 $\mathbf{k}_t^R = \mathrm{RoPE}(W^{KR}\mathbf{h}_t)$
  - 最终拼接使用：

$$\mathbf{q}_{t,i} = [\mathbf{q}_{t,i}^C; \mathbf{q}_{t,i}^R], \quad \mathbf{k}_{t,i} = [\mathbf{k}_{t,i}^C; \mathbf{k}_t^R]$$

- **缓存代价**：需额外缓存 $\mathbf{k}_t^R$，总 KV 缓存为 $(d_c + d_h^R)l$

性能与效率对比

注意力机制	每 token KV 缓存（元素数）	能力
MHA	$2n_h d_h l$	强
GQA	$2n_g d_h l$	中等
MQA	$2d_h l$	弱
MLA（本工作）	$(d_c + d_h^R)l \approx \frac{9}{2}d_h l$	**更强**

- **实际配置**：$d_c = 4d_h, d_h^R = d_h/2 \rightarrow$ 缓存 ≈ GQA with 2.25 groups
- **结论**：以接近 GQA 的缓存开销，实现**超越 MHA 的性能**

Multi-Token Prediction（MTP）

核心目标

在训练阶段，让模型在每个位置**同时预测多个未来 token**（共 D 个），以：

- **密集化训练信号**，提升数据效率；
- **引导模型预规划表征**，增强对后续 token 的预测能力。

关键设计：顺序因果预测（vs 并行独立预测）

- 采用 D **个顺序堆叠的 MTP 模块**，每层基于前一层表示和目标 token 嵌入进行预测，
- **严格保持因果链**：第 k 层预测依赖于第 $k-1$ 层输出，确保训练时信息流符合自回归约束。

模块结构（第 k 层）

对输入 token t_i，执行：

1. **特征融合**：

$$\mathbf{h}_i'^k = M_k \left[\mathrm{RMSNorm}(\mathbf{h}_i^{k-1});\ \mathrm{RMSNorm}(\mathrm{Emb}(t_{i+k}))\right]$$

   - $\mathbf{h}_i^{k-1}$：上一层表示（$k=1$ 时来自主模型）
   - $\mathrm{Emb}(t_{i+k})$：目标 token 的嵌入（来自主模型共享词表）
2. **Transformer 块处理**：

$$\mathbf{h}_i^k = \mathrm{TRM}_k(\mathbf{h}_i'^k)$$

3. **共享输出头预测**：

$$P_{i+k+1}^k = \mathrm{OutHead}(\mathbf{h}_i^k)$$

   - 输出头与主模型共享，节省参数并促进知识迁移。

训练目标

- 每层计算交叉熵损失 $\mathcal{L}_{\mathrm{MTP}}^k$
- 总 MTP 损失为加权平均：

$$\mathcal{L}_{\mathrm{MTP}} = \frac{\lambda}{D} \sum_{k=1}^{D} \mathcal{L}_{\mathrm{MTP}}^k$$

- 作为辅助损失加入主训练目标。

推理阶段

- **MTP 模块可完全丢弃**，主模型独立运行；
- 亦可**复用于推测解码**（speculative decoding），加速生成。

优势总结：在不增加推理负担的前提下，通过多步前瞻预测提升模型表征能力与训练效率。

DualPipe：高效流水线并行算法

动机

跨节点专家并行（EP）引入高通信开销（计算:通信 ≈ 1:1），传统 PP 方法气泡大、通信无法隐藏。

核心思想

- **双向流水调度**：从 pipeline 两端同时注入 micro-batch；
- **细粒度计算-通信重叠**：将每个 chunk 拆分为子操作，在 forward/backward 对之间交错执行，**完全隐藏 all-to-all 与 PP 通信**。

关键技术

- **Chunk 分解**：
  `attention` → `all-to-all dispatch` → `MLP` → `all-to-all combine`
  （反向中 `attention` / `MLP` 进一步拆为 “backward for input” 与 “backward for weights”）
- **SM 资源动态分配**：手动调节用于通信 vs 计算的 SM 比例；
- **双向调度示例**：8 PP rank + 20 micro-batch，实现高吞吐、低气泡。

性能优势（vs 1F1B / ZB1P / Chimera）

方法	流水气泡	参数内存	激活内存	约束条件
DualPipe	$(\frac{PP}{2}-1)(F\&B+B-3W)$	$2\times$	$PP+1$	PP 与 micro-batch 为偶数

- **气泡显著减少**
- **激活内存仅增** $1/PP$
- **通信完全隐藏**，支持大规模跨节点 MoE
- **扩展性好**：micro-batch 增加不影响气泡或内存

工程协同优化

- 与定制 **跨节点 all-to-all 通信内核** 协同，最大化 IB/NVLink 带宽利用率；
- 结合 **MTP 模块部署策略**（首尾层同 rank），实现嵌入/输出头物理共享，进一步节省内存。

> **优势总结**：DualPipe 在高通信负载场景下实现近零通信开销，同时保持低内存增长与高硬件利用率，是支撑高效训练的关键基础设施。

基座模型上的纯强化学习

核心目标

探索 LLM 是否能在**无监督数据**条件下，通过 RL 自主演化出强推理能力。

算法：Group Relative Policy Optimization (GRPO)

- **优势**：无需训练与策略模型同规模的 Critic，降低计算开销；
- **机制**：对每个问题 q，采样 G 个输出 $\{o_i\}$，利用组内奖励均值与标准差归一化计算优势 A_i；
- **优化目标**：

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}(\text{PPO-style clipped ratio} \cdot A_i - \beta \cdot \text{KL}(\pi_\theta \| \pi_{\text{ref}}))\right]$$

其中 KL 散度约束防止策略偏离参考模型过远。

奖励设计（纯规则驱动）

- **准确性奖励**：基于确定性答案验证（如数学题 boxed 答案、LeetCode 编译测试）；
- **格式奖励**：强制模型将推理过程置于 `<think>...</think>` 标签内，答案置于 `<answer>...</answer>`。
- **不使用神经奖励模型**：避免奖励黑客（reward hacking）与额外训练开销。

训练模板

采用极简指令模板，仅约束结构（先推理后答案），**不引入内容偏置**，以观察模型自然演化过程。

关键结果

性能表现

- 在 AIME 2024 上，pass@1 从 15.6% 提升至 **71.0%**，接近 OpenAI o1-0912；
- 多数投票（cons@64）下达 **86.7%**，**超越 o1-0912**；
- 在 MATH-500、GPQA Diamond 等基准上表现强劲（见表 \ref{tab:r1-zero}）。

自演化过程

- 模型**自发延长推理长度**，生成数百至数千 token 的思考过程；
- **涌现高级行为**：如自我反思、重试替代解法——**非显式编程，而是 RL 激励下的自然涌现**。

“Aha Moment”

- 中期模型学会在推理中途**主动回溯并修正错误**（见表 \ref{tab:aha_moment}）；
- 体现 RL 的核心魅力：**仅提供正确激励，模型自主发展复杂策略**。

局限性

- 输出可读性差、存在语言混杂等问题，不适合直接面向用户。

带冷启动的强化学习

为解决的可读性问题并进一步提升性能，引入**小规模高质量 CoT 数据作为 RL 起点**。

冷启动阶段

- 收集**数千条长 CoT 示例**，来源包括：
  - 少样本提示生成；
  - 可读输出筛选；
  - 人工后处理优化。
- 微调 DeepSeek-V3-Base，作为 RL 初始策略。
- **输出格式标准化**：`|token|<reasoning>|token|<summary>`，提升可读性与用户友好性。

推理导向的 RL 训练

- 沿用 \dsro{} 的 GRPO 框架；
- **新增语言一致性奖励**：鼓励 CoT 使用单一目标语言（虽轻微牺牲性能，但提升可读性）；
- 总奖励 = 准确性奖励 + 语言一致性奖励。

拒绝采样 + 监督微调（SFT）

RL 收敛后，进行第二轮 SFT，构建约 **800k 样本**数据集：

- **推理数据（~600k）**：
  - 从 RL 模型拒绝采样正确轨迹；
  - 引入生成式奖励模型（DeepSeek-V3 判定）扩展数据；
  - 过滤语言混杂、冗长或含代码块的 CoT。
- **非推理数据（~200k）**：
  - 复用 DeepSeek-V3 的 SFT 数据；
  - 对简单查询（如“hello”）不生成 CoT。

全场景对齐 RL

- **二次 RL 阶段**：兼顾推理、有用性与无害性；
  - **推理任务**：继续使用规则奖励；
  - **通用任务**：采用偏好对 + 奖励模型；
  - **评估策略**：
    - 有用性：仅评估 `<summary>`；
    - 无害性：评估完整响应（含推理过程）。

蒸馏：赋能小型模型

- **方法**：将上述 800k 高质量 CoT 数据用于 SFT 蒸馏；
- **目标模型**：Qwen2.5-Math（1.5B/7B）、Qwen2.5（14B/32B）、Llama-3.1-8B、Llama-3.3-70B-Instruct；
- **结果**：**仅 SFT 即显著提升小模型推理能力**；
- **定位**：展示蒸馏有效性，**RL 阶段留待社区探索**。